Report finale – Modelli DfpNet Mamba e tuning

18/10/2026
Fonte: `train/metrics/`

Sintesi

Il modello migliore è **HalfMamba_Augmented, Combination 4**: Mamba 16-2-4 su `out1`, `out2` e `out2b`, più un blocco **32-2-4** nel decoder a 64×64. Ha un errore di validazione dell'**1,628%** ($-16{,}8\%$ rispetto alla baseline) con il **+10,5% di parametri** e 30 s per epoca.

- **Miglior compromesso:** Combination 12 (`out2` a 24-2-2). Errore 1,673% ($-14{,}5\%$), +9,8% di parametri, 27,4 s per epoca.
- **Massima efficienza:** `M_out1`, un solo blocco Mamba a 64×64. Dà $-8{,}7\%$ con appena +1,2% di parametri e 15,8 s per epoca.
- **Il tuning conta soprattutto nel decoder e su `out2`.** Toccare `out1` non migliora mai il riferimento, e `out2b` a 32-2-2 è la scelta peggiore.
- **Limite:** tutte le run sono a 1 seed, e lo stesso modello cambia fino a 0,15 punti tra il seed 34 e il 42. Le differenze sotto $\sim$0,1 punti non sono affidabili.

1 Metodologia

Tutti i modelli sono confrontati con la stessa metrica, ricalcolata dai file `metrics_train.csv` di ogni run in `train/metrics/`.

- **Errore val (20):** media di `val_rel_error` nelle ultime 20 epoche su 150. È la metrica già usata da `runTuning.py` ed è più stabile dell'ultima epoca.
- **Baseline:** media dei 3 seed (42, 1955, 2003) = $\mathbf{1{,}958\% \pm 0{,}025}$. `runTuning.py` usa invece 1,953%, cioè il solo seed 42, quindi i suoi "improvement" sono leggermente diversi da quelli qui.
- **Δ parametri:** rispetto ai 2.332.035 parametri della baseline (`expo = 5`), contati con il codice attuale di `DfpNet.py`. Per le run storiche allenate con codice vecchio (es. `mambaSup` con `ssm_conv = 3`) il conteggio è approssimato.
- **Tempo e VRAM:** media dall'epoca 2 in poi. La baseline non li ha registrati. Come riferimento uso i $\sim$10,4 s per epoca dei modelli con attention a bassa risoluzione (`A_out2`…`A_out5`), che costano quasi quanto la baseline.
- **Riferimento del tuning:** `HMdec_d2`, cioè la stessa architettura di HalfMamba_Augmented con tutti i blocchi a 16-2-4 e lo stesso seed 42. Errore 1,733%.
- **Esclusa:** la run `HalfMamba_Augmented_Combination_{i}_s42` (1,686%). Il nome viene da un bug del vecchio script e non si sa quale configurazione abbia usato.

2 Panoramica di tutti i modelli

Sono stati provati 33 modelli fuori dal tuning: 24 riducono l'errore della baseline di più del 3%, e tutti i 14 migliori hanno Mamba a `out1` (64×64). M = blocco Mamba 16-2-4, m = Mamba small, A = self-attention. Δ errore negativo = meglio della baseline. Le 15 combinazioni del tuning sono nella Sezione 4.

Tabella 1: Risultati di validazione di tutti i modelli (ordinati per errore).

Modello	Mixer	Seed	Errore val (20)	Δ errore	Parametri	Δ param.	s/epoca	VRAM MB
`HMdec_d2`	halfMamba + dout2	42	1,733%	−11,5%	2,56 M	+10,0%	27,0	723
`HMdec_d2_d2b`	halfMamba + dout2, dout2b	42	1,739%	−11,1%	2,65 M	+13,8%	30,5	815
`HMdec_d2_d2b_d3`	halfMamba + dout2, dout2b, dout3	42	1,750%	−10,6%	2,74 M	+17,5%	32,0	840
`halfMamba_lightconv1`	M out1, out2, out2b	42	1,753%	−10,5%	2,54 M	+8,8%	23,4	501
`MMM_AA`	M out1–out2b; A out4, out5	42	1,761%	−10,0%	3,06 M	+31,4%	25,4	527
`halfMamba`	M out1, out2, out2b	42	1,777%	−9,2%	2,54 M	+8,8%	23,9	506
`M_out1`	M out1	42	1,788%	−8,7%	2,36 M	+1,2%	15,8	385
`halfMambaPush`	M out1, out2; A out2b	42	1,789%	−8,6%	2,47 M	+5,7%	21,4	415
`MambaMentality_convo`	M in, out1–out2b	42	1,795%	−8,3%	2,54 M	+8,9%	34,3	729
`MM___`	M out1, out2	42	1,800%	−8,1%	2,45 M	+5,0%	19,1	479
`Mmm___`	M out1; m out2, out2b	42	1,813%	−7,4%	2,44 M	+4,5%	22,3	446
`MMAA__`	M out1, out2; A out2b, out3	42	1,820%	−7,0%	2,53 M	+8,6%	22,8	497
`fullMamba`	M out1, out2; m out2b–out5	42	1,822%	−6,9%	3,66 M	+57,1%	35,3	515
`halfMambaPush_Light`	M out1, out2; A out2b	42	1,828%	−6,6%	2,47 M	+5,7%	20,2	409
`A_out2`	A out2	42	1,859%	−5,0%	2,35 M	+0,7%	10,5	207
`A_out1`	A out1	42	1,870%	−4,5%	2,34 M	+0,2%	105,5	10324
`M_out2`	M out2	42	1,876%	−4,2%	2,42 M	+3,8%	13,0	270
`HalfMamba_Augmented_exp1`	M out1–out2b + dout2	34	1,881%	−3,9%	2,56 M	+10,0%	27,3	676
`M_out3`	M out3	42	1,882%	−3,9%	2,63 M	+12,8%	12,8	205
`M_out5`	M out5	42	1,882%	−3,8%	3,42 M	+46,7%	12,8	210
`M_out2b`	M out2b	42	1,886%	−3,6%	2,42 M	+3,8%	12,8	201
`MMMA__`	M out1–out2b; A out3	42	1,889%	−3,5%	2,60 M	+11,6%	25,1	517
`A_out3`	A out3	42	1,896%	−3,2%	2,40 M	+2,8%	10,3	209
`HalfMamba_Augmented_h24r2mlp6d`	M out1–out2b + dout2	34	1,896%	−3,2%	2,56 M	+10,0%	28,5	751
`A_out2b`	A out2b	42	1,906%	−2,6%	2,35 M	+0,7%	10,4	195
`MambaHead`	M huge out1; M out2, out2b	34	1,911%	−2,4%	2,59 M	+11,0%	30,9	676
`MambaMentality`	M in, out1–out2b	34	1,915%	−2,2%	2,54 M	+8,9%	33,2	728
`M_out4`	M out4	42	1,920%	−1,9%	3,42 M	+46,7%	13,0	126
`A_out4`	A out4	42	1,927%	−1,6%	2,60 M	+11,3%	10,5	210
`A_out5`	A out5	42	1,932%	−1,3%	2,60 M	+11,3%	10,4	208
`mambaSup`	M out2b	3 seed	1,933% ± 0,057	−1,3%	2,42 M	+3,8%	–	–
`baseline`	nessuno	3 seed	1,958% ± 0,025	–	2,33 M	–	–	–
`M1M3`	M out1, out2b	34	2,044%	+4,4%	2,45 M	+5,0%	19,5	408

I 5 modelli allenati con il seed 34 stanno tutti nella metà bassa della classifica. HalfMamba_Augmented ha la stessa architettura di `HMdec_d2`, ma con il seed 34 fa 1,881% invece di 1,733%. Questa differenza da sola (0,15 punti) è più grande di quella che separa i primi 14 modelli.

3 Migliori modelli

Aggiungere parametri non basta: a parità di +10% di parametri l'errore va dal −3,9% al −16,8%. Quello che conta è dove si mette Mamba e come è configurato. Sono i migliori i modelli sulla linea blu della Figura 1, perché nessun altro modello fa meglio con meno parametri.

Tabella 2: I quattro modelli migliori secondo prestazioni, parametri e tempo.

#	Modello	Perché	Errore val (20)	Δ param.	s/epoca	VRAM MB
1	Combination 4	Miglior errore in assoluto	1,628% (−16,8%)	+10,5%	30,0	744
2	Combination 12	Miglior compromesso	1,673% (−14,5%)	+9,8%	27,4	722
3	`M_out1`	Miglior guadagno per parametro	1,788% (−8,7%)	+1,2%	15,8	385
4	`halfMamba_lightconv1`	Il più veloce tra quelli sopra il −10%	1,753% (−10,5%)	+8,8%	23,4	501

- **Combination 4** è il migliore perché rinforza il blocco Mamba del decoder a 64×64 (`d_state` da 16 a 32), dove la rete ricostruisce la scia. È l'unico modello che abbassa in modo visibile anche la pressione: 8,80% contro il 9,70% della baseline (circa 1,5 deviazioni standard). Il costo è circa 3 volte il tempo della baseline (30 s contro ∼10 s).

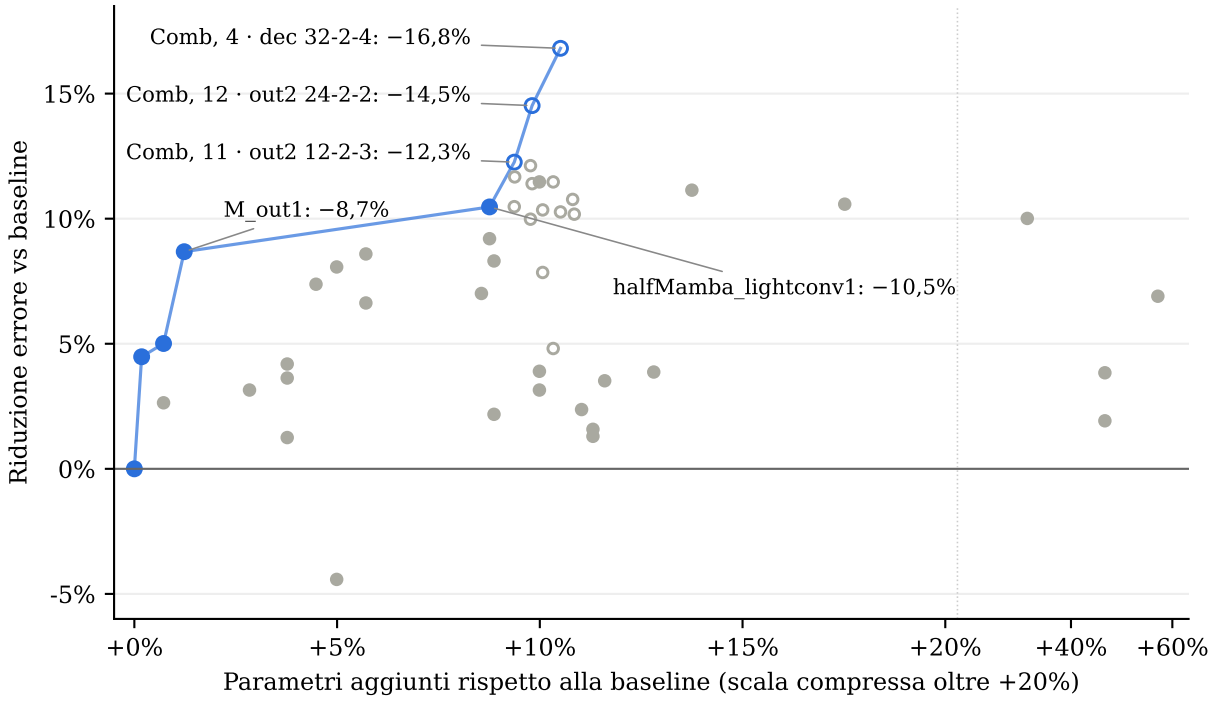


Figura 1: Combination 4 riduce l'errore del 16,8% con il 10,5% di parametri in più. Riduzione dell'errore di validazione rispetto alla baseline per ogni modello. La linea blu unisce i modelli non dominati. Cerchi pieni: architetture provate. Cerchi vuoti: combinazioni del tuning di HalfMamba_Augmented.

- **Combination 12** perde solo 0,045 punti rispetto alla 4, ma ha meno parametri, è il 9% più veloce e usa 22 MB in meno. È la scelta giusta se il tempo di training conta.
- `M_out1` ottiene metà del guadagno della Combination 4 con un decimo dei parametri aggiunti (+28.576) e metà del tempo. Rende circa 7 punti di riduzione dell'errore per ogni 1% di parametri, contro 1,6 della Combination 4.
- **`halfMamba_lightconv1`** è il punto di partenza dei modelli successivi. Aggiungere il blocco del decoder (`HMdec_d2`) costa +1,2% di parametri e +3,6 s per epoca per un punto in più di miglioramento.

Da evitare. `fullMamba` (+57% di parametri, 35 s/epoca, solo −6,9%), `MMM_AA` (+31% per lo stesso risultato di halfMamba), `M_out4` e `M_out5` (+47% per −2/ − 4%), `A_out1` (105 s/epoca e 10,3 GB per −4,5%). Anche altri blocchi nel decoder (`HMdec_d2_d2b`, `_d3`) aggiungono fino al 17,5% di parametri e 5 s/epoca senza migliorare.

4 Tuning degli iperparametri Mamba

Su 15 combinazioni solo 2 migliorano il riferimento in modo chiaro: il decoder a 32-2-4 (−0,105 punti) e `out2` a 24-2-2 (−0,060). Ogni configurazione è scritta come `d_state-ssm_ratio-mlp_ratio`, e ogni combinazione cambia un solo blocco rispetto a 16-2-4.

- **Decoder `dout2`: è il blocco più sensibile** (0,18 punti tra il migliore e il peggiore). Alzare `d_state` a 32 tenendo `mlp_ratio` 4 dà il miglior risultato di tutto il progetto. Con `mlp_ratio` 2 (24-2-2) invece peggiora, quindi qui l'MLP serve. Il 16-1-2 resta pari al riferimento ma è il più leggero: 24,9 s/epoca e 559 MB di VRAM.

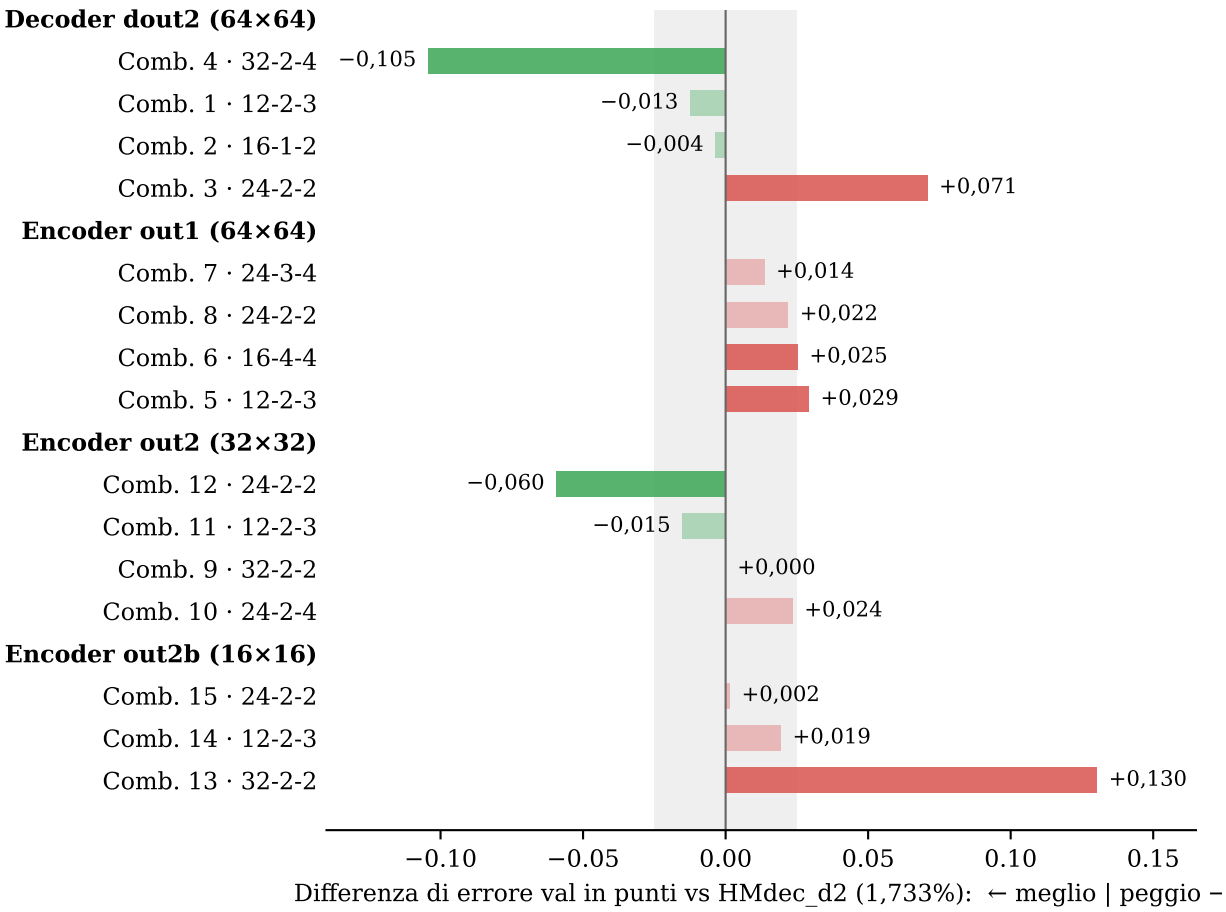


Figura 2: Solo due modifiche battono il riferimento oltre il rumore. Differenza dell'errore di validazione in punti rispetto a `HMdec_d2` (tutti i blocchi 16-2-4, seed 42). Fascia grigia: ±1 deviazione standard della baseline tra seed. Barre chiare: differenza dentro la fascia di rumore.

- **Encoder `out1`: qualsiasi modifica peggiora** (da +0,014 a +0,029). Le varianti più grandi costano anche di più: 16-4-4 arriva a 31,6 s e 859 MB. Va lasciato a 16-2-4.
- **Encoder `out2`: conviene ridurre l'MLP.** 24-2-2 e 12-2-3 migliorano con meno parametri, mentre 24-2-4 peggiora.
- **Encoder `out2b`: `d_state` alto fa male.** Con 32-2-2 si ha il risultato peggiore del tuning (+0,130). A 16 × 16 la sequenza è di soli 256 token e uno stato grande non serve. 24-2-2 è neutro.

Due cose sono comuni a tutto il tuning. Il guadagno arriva dal ritoccare singoli blocchi, non dall'aumentare la capacità: tutte le combinazioni hanno tra +9,4% e +10,9% di parametri. E le differenze sotto ∼0,03 punti non vanno lette come effetti reali.

5 Configurazione consigliata

La configurazione che consiglio è la **Combination 4: 16-2-4 / 16-2-4 / 16-2-4 / 32-2-4** (`out1` / `out2` / `out2b` / decoder). La prossima run dovrebbe però provare a unirla alla Combination 12. Perché la Combination 4:

- **Prestazioni:** è la migliore su tutte e tre le componenti (p, v_x, v_y). Il guadagno sul riferimento (0,105 punti) è circa 4 volte la deviazione standard della baseline tra seed.

Criterio	Combination 4	Combination 12	Riferimento (tutti 16-2-4)
Errore val (20)	**1,628%**	1,673%	1,733%
Δ vs baseline	**−16,8%**	−14,5%	−11,5%
Errore pressione p	**8,80%**	9,11%	9,29%
Δ parametri	+10,5%	**+9,8%**	+10,0%
Tempo per epoca	30,0 s	27,4 s	**27,0 s**
VRAM	744 MB	**722 MB**	723 MB

- **Parametri:** costa solo lo 0,5% in più del riferimento. Rispetta il vincolo di iso-capacità come tutti i modelli halfMamba.
- **Tempo:** +3 s per epoca (+11%), cioè circa 7 minuti in più su 150 epoche. È un costo piccolo per il miglior risultato del progetto.

Se conta di più il tempo, la Combination 12 dà l'86% del guadagno ed è la più economica tra le migliori. Se contano soprattutto i parametri, resta `M_out1`.

Passi successivi

- ☐ Allenare la combinazione unita **16-2-4 / 24-2-2 / 16-2-4 / 32-2-4**. Le due modifiche toccano blocchi diversi e potrebbero sommarsi (circa +10,3% di parametri).
- ☐ Rilanciare Combination 4, Combination 12 e la combinazione unita con 3 seed (42, 1955, 2003). Con un solo seed lo stesso modello oscilla fino a 0,15 punti.
- ☐ Provare `d_state` 48 o 64 sul decoder: è l'unico blocco in cui alzare `d_state` ha aiutato.
- ☐ Valutare i modelli finali sul test set con `runTest.py`. Oggi solo la baseline ha un risultato sul test.
- ☐ Registrare tempo e VRAM della baseline, per confrontare i costi in modo diretto.